



M S RAMAIAH COLLEGE OF ARTS, SCIENCE AND COMMERCE

DEPARTMENT OF COMPUTER SCIENCE

A Project Report on

“ORGAN DONATION PROCUREMENT”

Submitted in partial fulfilment of the requirement for the award degree of

BACHELOR of COMPUTER APPLICATION

IN

BENGALURU CITY UNIVERSITY

By

Name: GAGANASHREE C

Reg No: U18EV21S0079

**Under the guidance of
Mrs. SHAIK VALLI HASEENA**



RAMAIAH
College of Arts, Science
& Commerce

M S RAMAIAH COLLEGE OF ARTS, SCIENCE AND COMMERCE

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that **"GAGANASHREE C (U18EV21S0079)"** has successfully completed the project titled **"ORGAN DONATION PROCUREMENT"** for the 6th Semester, as partial fulfilment of the requirements for the completion of Degree in **Bachelor of Computer Applications**, awarded by the **Bengaluru City University** for the Academic

GUIDE

Mrs. SHAIK VALLI HASEENA

HOD

Mr. BASAVARAJ C M

External Examiners

1. _____

2. _____

Examination Centre: _____

Date:

M S RAMAIAH COLLEGE OF ARTS, SCIENCE AND COMMERCE

DECLARATION

The project titled “**ORGAN DONATION PROCUREMENT**” is developed by me in the partial fulfilment of **Bengaluru City University**. It is a systematic work carried by me under the guidance of **Mrs. SHAIK VALLI HASEENA**, Assistant Professor in Computer Science Department,

M S RAMAIAH COLLEGE OF ARTS, SCIENCE AND COMMERCE, Bengaluru-560054

I declare that this project has not been submitted to any Degree or Diploma to Bengaluru City University or any other University.

SUBMITTED BY:

GAGANASHREE C (U18EV21S0079)

DATE: _____

SIGNATURE: _____

ACKNOWLEDGEMENT

The department of software is generally a bit complex and time consuming. The goal of developing the project “Organ Donation Procurement” could not be achieved without the encouragement of kindly helpful and supportive people. Hereby we convey our sincere thanks for all of them. We take this opportunity to express my gratitude to people who had been instrumental in the successful completion of this project we are thankful to our management trustee for providing us an opportunity to work and complete the project successfully We wish to express my thanks to our principal for her support to the project work. We would like to acknowledge my gratitude to our HOD of the Department of Computer Science Basavaraj C M for her encouragement and support. Without her encouragement and guidance this project would not have materialized.

ABSTRACT

Project Title: Organ Donation Procurement

Description:

Organ procurement (also called surgical recovery) is a surgical procedure that removes organs or tissues for reuse, typically for organ transplantation. Organ transplantation is a medical procedure in which an organ is removed from one body and placed in the body of a recipient, to replace a damaged or missing organ. The donor and recipient may be at the same location, or organs may be transported from a donor site to another location. Organs and/or tissues that are transplanted within the same person's body are called allografts. Transplants that are recently performed between two subjects of the same species are called allografts. Allografts can either be from a living or cadaveric source.

Transplantation medicine is one of the most challenging and complex areas of modern medicine. Some of the key areas for medical management are the problems of transplant rejection, during which the body has an immune response to the transplanted organ, possibly leading to transplant failure and the need to immediately remove the organ from the recipient. When possible, transplant rejection can be reduced through serotyping to determine the most appropriate donor-recipient match and through the use of immunosuppressant drugs.

Organ donors may be living or may have died of brain death or circulatory death. Most deceased donors are those who have been pronounced brain dead. Brain dead means the cessation of brain function, typically after receiving an injury (either traumatic or pathological) to the brain, or otherwise cutting off blood circulation to the brain (drowning, suffocation, etc.). Breathing is maintained via artificial sources, which, in turn, maintains heartbeat. Once brain death has been declared, the person can be considered for organ donation.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
Chapter 1	Introduction	5
Chapter 2	Requirement specifications	8-13
2.1	Hardware requirements	
2.2	Software requirements	
2.3	Tools and technologies	
Chapter 3	Description	14-21
3.1	ER diagram	
3.2	Schema diagram	
Chapter 4	Creating tables	22-35
4.1	Inserting tables	
4.2	Queries	
Chapter 5	Coding	35-47
Chapter 6	Snapshots	48-54
6.1	Conclusion	
6.2	Bibliography	

CHAPTER 1

INTRODUCTION

Overview

This report discusses the result of the work done in development of "Organ donation and Procurement" on "HTML" and "PHP" Front-end Platform and "My SQL" as back-end Platform. At the development of an application PHP provides a good connecting facility between all pages, also the back-end My SQL is most important to save all the data related the application.

This report discusses the result of the work done in the development of "Organ Donation and Procurement" on an "HTML" and "PHP" front-end platform and "MySQL" as the back-end platform. At the development of an application, PHP provides a good connecting facility between all pages. The back-end MySQL is also most important to save all the data related to the application.

Organ donation and procurement is a critical area in healthcare, aiming to address the gap between the demand and supply of organs for transplantation. This application is designed to streamline and enhance the process, making it more efficient and accessible for donors, recipients, and healthcare professionals. By using HTML and PHP for the front-end development, the application ensures a user-friendly interface, which facilitates easy navigation and interaction.

Background and Motivation

The core issue we address involves transitioning from a manual system to a fully automated system.

Manual System

The existing manual system is extremely time-consuming and inefficient. It is prone to errors and often lacks a structured approach to solving various problems. Manual processes can lead to delays, miscommunications, and inaccuracies, which are particularly detrimental in the context of organ donation and procurement, where timely and precise information is crucial.

Technical System

With the rapid advancement of technology, failing to update our systems can result in significant losses over time. Modern technical systems incorporate the latest tools and technologies, such as computers, printers, fax machines, and the Internet. These systems are designed to be fast, accurate, user-friendly, and reliable. By leveraging these technologies, we can ensure that our operations are streamlined and efficient.

Transitioning to a technical system offers numerous benefits. Automated systems reduce the risk of human error, enhance data accuracy, and facilitate real-time data processing. This is particularly important in organ donation and procurement, where timely decisions can save lives.

Additionally, a technical system can integrate various functions, such as donor registration, matching donors with recipients, and tracking the entire donation process, from initial registration to post-transplant follow-up.

Implementing an automated system not only improves efficiency and accuracy but also enhances the overall user experience. It allows healthcare professionals to focus more on patient care rather than administrative tasks, and it provides donors and recipients with a seamless and transparent process. The adoption of a technical system is motivated by the need to provide better service, increase the number of successful transplants, and ultimately save more lives.

Objective

Need of Organ Donation Websites:

- To promote organ donation for transplantation as a treatment for many life threatening diseases including heart disease, kidney disease, liver disease, diabetes and cystic fibrosis
- To educate and inform the community, patients and their families and health professionals about organ and tissue donation to markedly improve rates of donation.
- To work in partnership with Department of Health (DOH), clinicians, and hospitals to promote best practice professional training and ensure that the family of every potential donor is offered the option of donation in a caring and respectful manner.
- To provide support, care, information and advocacy for people and with end stage organ failure, donor families, living donors transplant recipients and their families.
- To provide policy advice to DOH, clinicians and hospitals.
- Assuring compliance with all external regulatory bodies, including but not limited to: the Organ Procurement and Transplantation Network (OPTN), the United Network for Organ Sharing (UNOS), Centers for Medicare and Medicaid Services (CMS) Conditions of Participation (COP), the Missouri State Department of Health (DOH), The Joint Commission (TJC) Standards
- Ensuring the programs accreditation
- Identifying opportunities for improvement
- Prioritizing performance improvement and patient safety projects within organ transplantation
- To promote organ donation for transplantation* as a treatment for many life-threatening diseases including heart disease, kidney disease, liver disease, diabetes, and cystic fibrosis. These platforms can significantly boost awareness and encourage more people to consider organ donation as a life-saving act.
- To educate and inform the community, patients and their families, and health professionals* about organ and tissue donation to markedly improve rates of donation. Providing comprehensive information helps dispel myths and misconceptions about organ donation, fostering a more informed and supportive community.
- To work in partnership with the Department of Health (DOH), clinicians, and hospitals* to promote best practice professional training and ensure that the family of every potential donor is offered the option of donation in a caring and respectful manner. Collaborative efforts ensure a streamlined and compassionate approach to organ donation.
- To provide support, care, information, and advocacy* for people with end-stage organ failure, donor families, living donors, transplant recipients, and their families. By offering a

robust support system, these websites help individuals navigate the complex emotional and logistical aspects of organ donation and transplantation.

- Furthermore, organ donation websites can facilitate the registration process for potential donors, provide updates on new developments in transplantation technology and procedures, and create a community of support and shared experiences for those affected by organ donation and transplantation. This holistic approach not only saves lives but also enhances the quality of life for recipients and their families.

CHAPTER 2

REQUIREMENT SPECIFICATION

2.1 Hardware Requirements

- Operating System: -windows 10
- Processor:- i5
- Processor Speed:- 3.4 GHZ
- Installed Memory(RAM):- 8 GB
- Hard Disk:- 500 GB
- System Type: - 64-bit OS

2.2 Software Requirements

- Front End :- HTML/CSS/PHP
- Back End :- XAMPP

2.3 Tools and Technology used

1.XAMPP

XAMPP is a free and open-source software package that makes it easy to set up a web server. Developed by Apache Friends, XAMPP stands for Cross-Platform (X), Apache (A), MariaDB (M), PHP (P), and Perl (P). It includes the Apache HTTP Server, the MariaDB database, and interpreters for PHP and Perl programming languages.

Usage of XAMPP

Local Development: XAMPP is widely used by developers to create and test web applications locally on their own computers before deploying them to a live server. This allows developers to work offline and ensures that their applications are functioning correctly before going live.

Easy Installation: XAMPP simplifies the process of setting up a web server. It comes with all the necessary components pre-configured, so users don't have to manually install and configure each component separately.

Database Management: With MariaDB (a fork of MySQL) included, XAMPP provides a robust database management system. Developers can create, manage, and interact with databases easily using tools like phpMyAdmin, which is also included in the XAMPP package.

Script Interpretation: XAMPP includes interpreters for PHP and Perl, allowing developers to run scripts written in these languages. This is essential for developing dynamic web applications that can interact with databases, handle forms, and perform other server-side tasks.

Cross-Platform Compatibility: XAMPP works on various operating systems, including Windows, macOS, and Linux. This cross-platform compatibility makes it a versatile tool for developers working in different environments.

Testing Environment: XAMPP provides a safe and isolated environment for testing new features or updates before they are applied to a live server. This helps prevent potential

2.SQL

SQL, or Structured Query Language, is used to communicate with databases. It helps store, retrieve, and manage data. Here are key points about SQL:

Query Data: Ask questions about data, e.g., `SELECT Name FROM Users`.

Add Data: Insert new data, e.g., `INSERT INTO Users (Name, Age) VALUES ('John', 25)`.

Update Data: Change existing data, e.g., `UPDATE Users SET Age = 26 WHERE Name = 'John'`.

Delete Data: Remove data, e.g., `DELETE FROM Users WHERE Name = 'John'`.

Create Tables: Organize data, e.g., `CREATE TABLE Users (ID INT, Name VARCHAR(100), Age INT)`.

Modify Tables: Change table structure, e.g., `ALTER TABLE Users ADD Email VARCHAR(100)`.

Remove Tables: Delete tables, e.g., `DROP TABLE Users`.

Combining Data from Multiple Tables

Joins: Combine data, e.g., `SELECT Users.Name, Orders.OrderID FROM Users JOIN Orders ON Users.ID = Orders.UserID`.

Built-in Functions: Perform calculations or modify data, e.g., `COUNT(*)` or `UPPER(Name)`.

Transactions: Ensure multiple operations are completed successfully, e.g., transferring money between accounts.

Permissions: Control who can see or change data, e.g., `GRANT SELECT ON Users TO alice`.

SQL is powerful, widely used, and easy to learn. It handles large amounts of data efficiently and is essential for anyone working with databases.

3.HTML

HTML, or HyperText Markup Language, is the core language for creating and designing web pages. It structures content and enhances the appearance and usability of websites. Here are key points about HTML:

Structure Content: HTML organizes content into elements like headings, paragraphs, lists, and tables, providing a clear structure for web pages.

Embed Multimedia: HTML enables the inclusion of images, videos, and audio files, making web pages more dynamic.

Create Links: HTML allows the creation of hyperlinks, facilitating easy navigation between web pages.

Forms and Inputs: HTML provides forms, input fields, buttons, and checkboxes for user interaction and data collection.

Styling with CSS: HTML is used with CSS (Cascading Style Sheets) to style web pages, including colors, fonts, and layouts.

Responsive Design: HTML supports responsive design, ensuring web pages look good on all devices.

Ease of Use: HTML provides an easy-to-use graphical user interface (GUI), making it accessible for developers and users.

HTML is essential for creating structured, visually appealing, and user-friendly web pages.

3.CSS

CSS, or Cascading Style Sheets, is a language used to enhance the design and presentation of HTML documents. It allows developers to control the appearance of web pages by specifying the style, layout, and formatting of elements. Here's more about CSS:

Styling HTML: CSS is used to define how HTML elements should be displayed. It can change colors, fonts, margins, padding, and other visual aspects to create a cohesive design.

External Stylesheets: CSS can be written in external files, separate from HTML documents. This makes it easier to maintain and update styles across multiple web pages.

Selectors: CSS uses selectors to target specific HTML elements for styling. Selectors can target elements by their tag names, classes, IDs, attributes, and more.

Responsive Design: CSS supports responsive design principles, allowing developers to create layouts that adjust and adapt to different screen sizes and devices.

Animations and Effects: CSS includes capabilities for animations, transitions, and other visual effects. This can enhance user experience and engagement on websites.

Modularity: By separating content (HTML) from presentation (CSS), developers can make websites more modular and easier to maintain.

Compatibility: CSS is compatible with all major web browsers, ensuring consistent styling across different platforms.

Integration: CSS integrates seamlessly with HTML and JavaScript, forming the core trio of web development technologies.

CSS is crucial for creating visually appealing and user-friendly web pages. It allows developers to transform basic HTML structures into attractive and functional websites. By leveraging CSS, developers can achieve a consistent design language, improve usability, and create engaging user interfaces.

4.PHP

PHP, which stands for PHP: Hypertext Preprocessor, is a versatile open-source scripting language widely used for web development.

It seamlessly integrates with HTML, making it easy to embed dynamic content within web pages. Here's more about PHP:

Database Interaction: PHP excels in database connectivity. It supports various databases like MySQL, PostgreSQL, SQLite, and more, allowing developers to retrieve, insert, update, and delete data from databases dynamically.

Form Handling: PHP simplifies the handling of HTML forms. It can collect form data, validate input, and send emails, making it essential for interactive web applications.

File Handling: PHP can handle file uploads, read and write files on the server, and manipulate file contents, providing powerful capabilities for managing website resources.

Integration: PHP code is embedded directly into HTML pages using special delimiters (`<? php ... ?>`), allowing developers to mix PHP logic with HTML markup seamlessly.

Dynamic Content: By embedding PHP within HTML, developers can create dynamic web pages that display different content based on user input, session state, or database queries.

PHP is widely adopted for its simplicity, flexibility, and robustness in web development. It powers a vast majority of websites and web applications, from small personal blogs to large enterprise systems. Its ability to handle complex tasks efficiently, coupled with extensive community support and a rich ecosystem of frameworks and libraries, makes PHP a preferred choice for developers worldwide.

5.FLASK

The Flask tool is utilized primarily for building web applications in Python. It is renowned for its simplicity and flexibility, making it ideal for developing small to medium-sized applications quickly. Flask provides features like URL routing, template rendering, and handling HTTP requests, enabling developers to create robust web applications efficiently. Additionally, Flask supports the integration of third-party libraries, making it adaptable to various development needs and enhancing its utility in building scalable web solutions.

Python Web Framework: Flask is a popular framework used for developing web applications in Python.

Simplicity and Flexibility: Known for its straightforward design and flexible architecture, Flask is ideal for quickly prototyping and building small to medium-sized projects.

Core Features: Flask includes essential features like URL routing, which directs incoming web requests to specific functions, and template rendering, enabling dynamic HTML generation.

HTTP Request Handling: It excels in handling HTTP requests and responses, facilitating smooth communication between users and web applications.

Modularity and Extensions: Flask supports the integration of third-party libraries and extensions, allowing developers to enhance functionality and adapt to various development needs.

Scalability: Its lightweight nature and modular structure make it suitable for building scalable web solutions by leveraging a robust ecosystem of plugins and community contributions.

User-Friendly Development: By abstracting complex web development tasks, Flask enables developers to focus on implementing business logic and improving user experience efficiently.

Community and Support: Backed by a vibrant community, Flask benefits from ongoing development, updates, and a wealth of resources for developers of all skill levels.

6.JAVASCRIPT

The need for JavaScript arises from its essential role in enhancing the functionality, interactivity, and user experience of web pages. Here are the key reasons why JavaScript is indispensable in web development:

1. **Client-Side Interactivity:** JavaScript enables dynamic and interactive elements on web pages, such as form validation, animations, and interactive maps, without needing to reload the entire page.
2. **Enhanced User Experience:** By allowing developers to create responsive and engaging interfaces, JavaScript significantly improves the user experience (UX), making websites more intuitive and user-friendly.
3. **DOM Manipulation:** JavaScript interacts with the Document Object Model (DOM) of web pages, enabling developers to manipulate and update page content dynamically based on user actions or events.
4. **Asynchronous Programming:** JavaScript supports asynchronous programming, which allows web applications to perform tasks such as fetching data from servers or handling multiple operations simultaneously without blocking the user interface.
5. **Cross-Browser Compatibility:** JavaScript is supported by all major web browsers, ensuring consistent functionality and behaviour across different platforms and devices.
6. **Support for Frameworks and Libraries:** JavaScript has a vast ecosystem of frameworks (e.g., React, Angular) and libraries that simplify complex tasks and accelerate development of modern web applications.
7. **Server-Side Development:** With platforms like Node.js, JavaScript can also be used for server-side development, enabling full-stack development using a single programming language.
8. **Community and Resources:** JavaScript benefits from a large and active developer community, offering extensive resources, tutorials, and open-source libraries that support continuous learning and problem-solving.

JavaScript is crucial in modern web development for creating interactive, responsive, and feature-rich web applications that enhance usability and deliver a superior user experience.

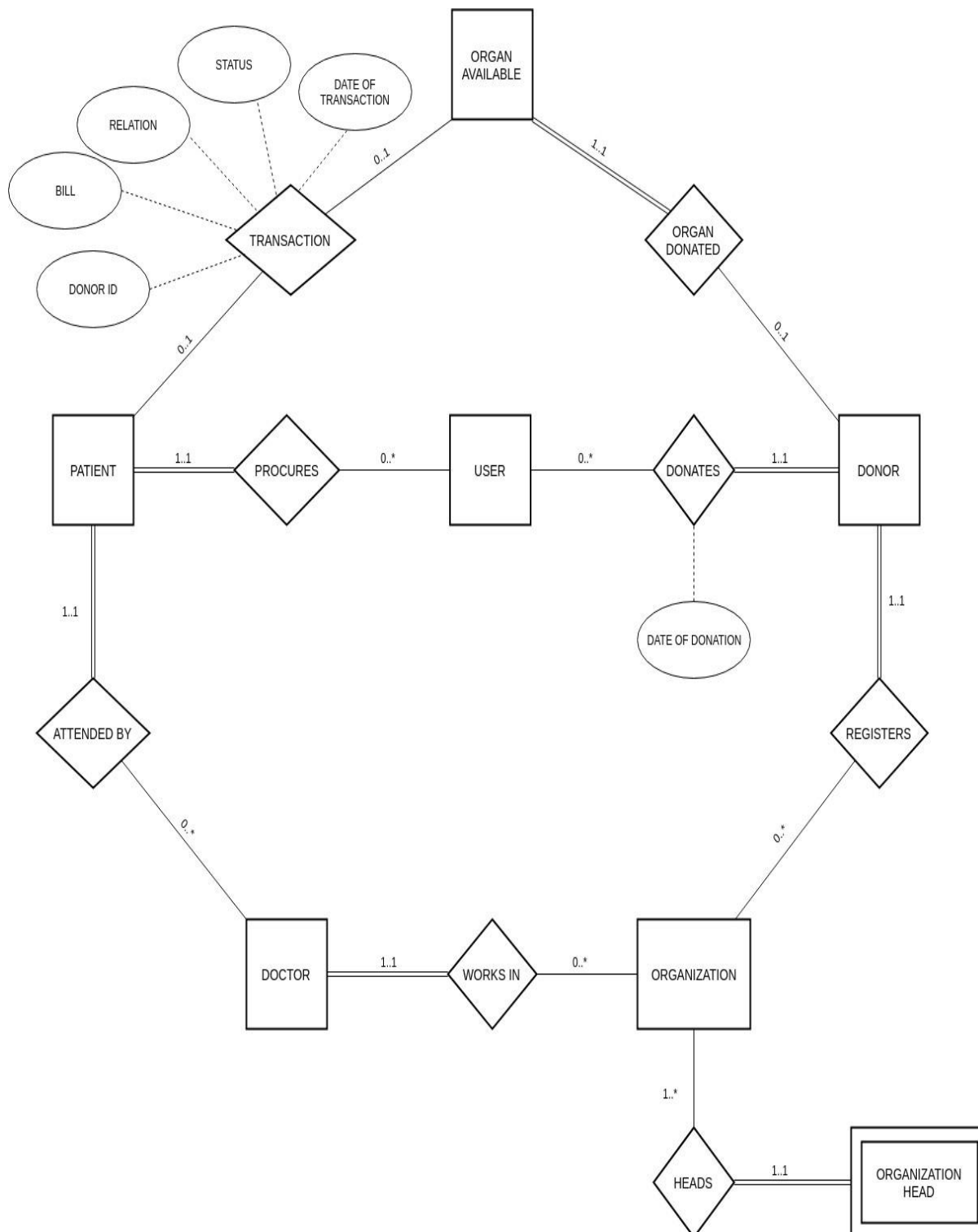
CHAPTER 3

DESCRIPTION

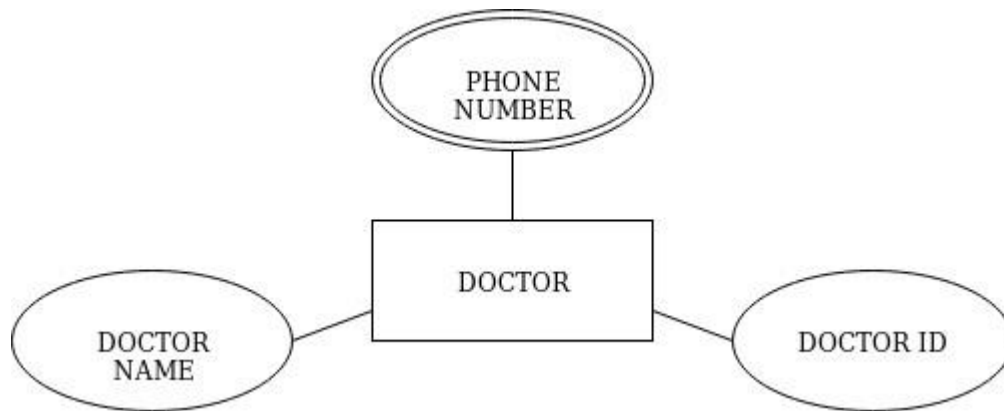
3.1 ER DIAGRAM:

An entity-relationship model (ER model) is a data model for describing the data or information aspects of a business domain or its process requirements, in an abstract way that lends itself to ultimately being implemented in the database such as a relational database. The main components of ER model are entities (things) and the relationship that can exist among them. Entity – relationship modeling was developed by Peter Chen and published in 1976 paper. The ER diagram is drawn to have a better understanding of the whole scenario, it is used to conceptualize the phenomena, actions, and interactions between various entities and to arrive at the specific requirements in a comprehensive manner. An entity relationship model is the result of using a systematic process to describe and define a subject area of business data. The data is represented as components (entities) that are linked with each other by relationships that express the dependencies and requirements between them, such as: one building may be divided into number of apartments, but one apartment can be located in one building. Entities may have various properties (attributes) that characterize them. Diagrams created to represent these entities, attributes, and relationships graphically are called entity-relationship diagrams. An ER model is typically implemented as database. In the case of a relational database, which stores data in tables, every row of each table represents one instance of an entity. Some data fields in these tables to indexes in other tables; such pointers are physical implementation of the relationships. The three- schema approach to software engineering uses three levels of ER models may be developed.

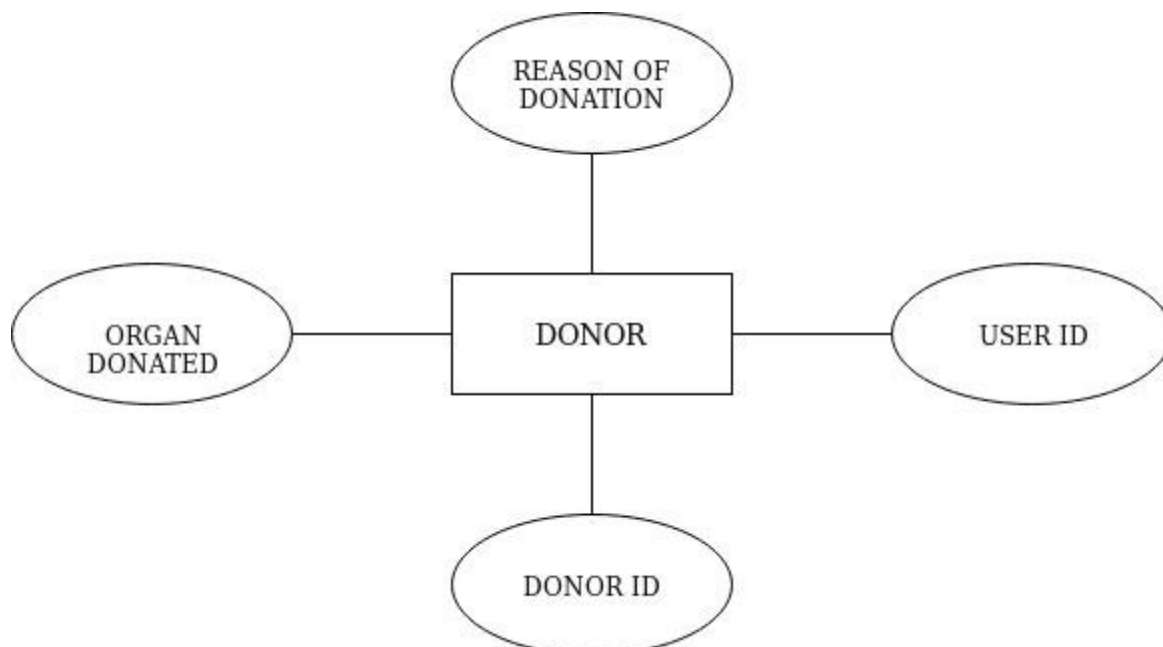
A relationship captures how entities are related to one another. Relationships can be thought of as verbs, linking two or more nouns. Entities and relationships can both have attributes. The ER diagram for this database is as shown:



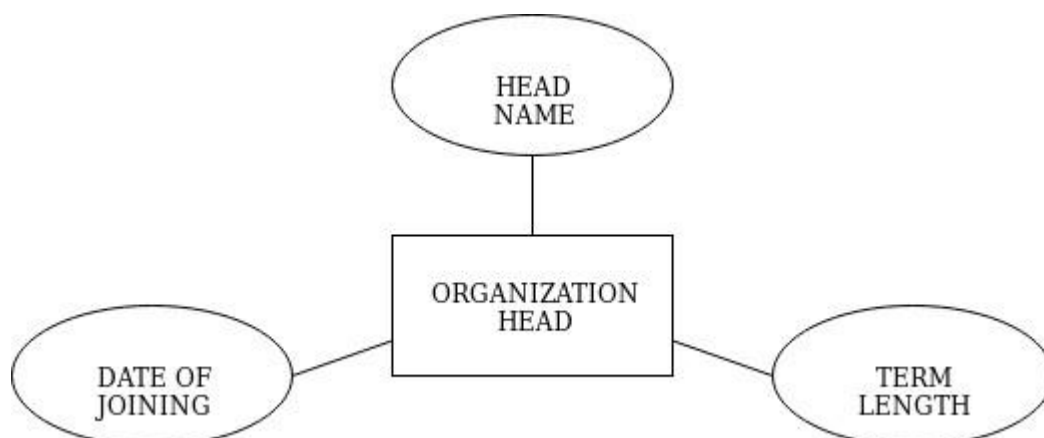
DOCTOR:



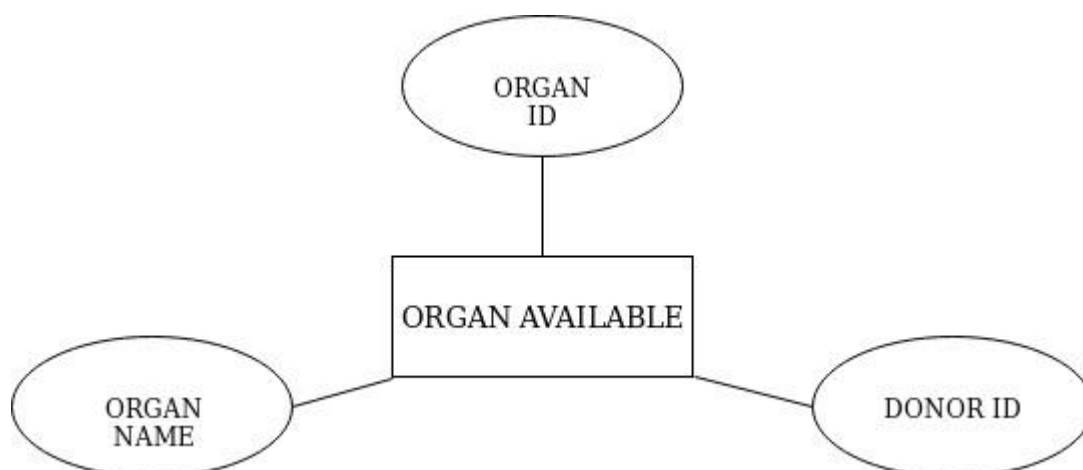
DONAR:



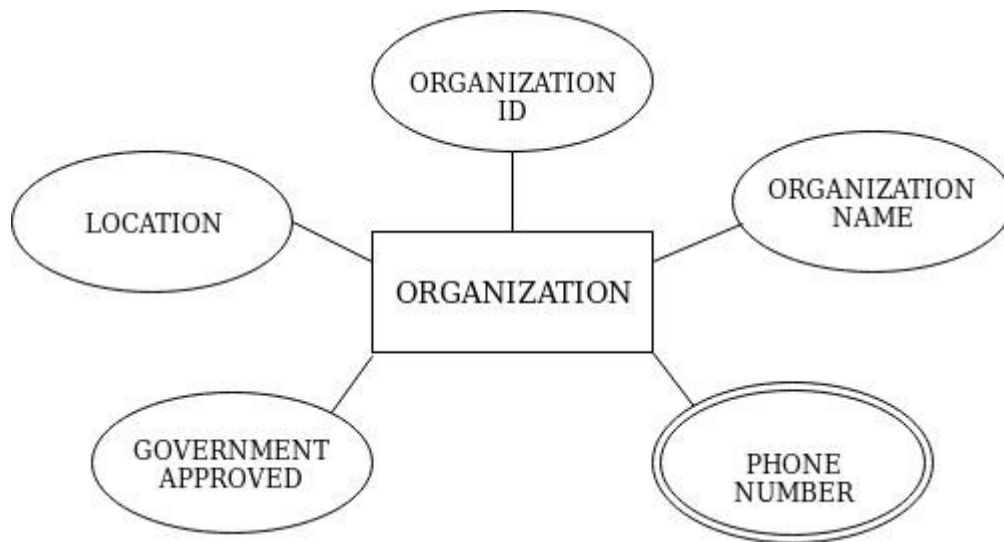
ORGANIZATION HEAD:



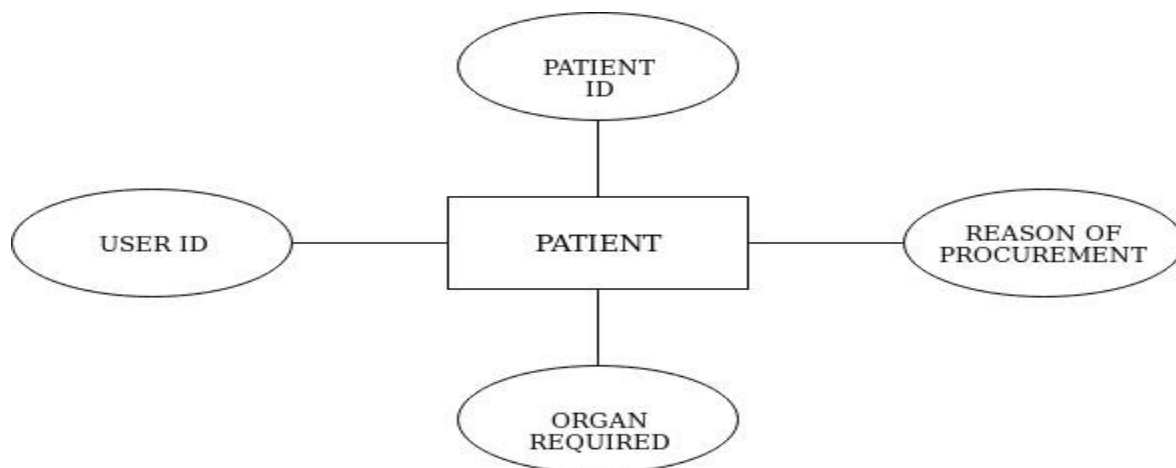
ORGAN AVAILABLE:



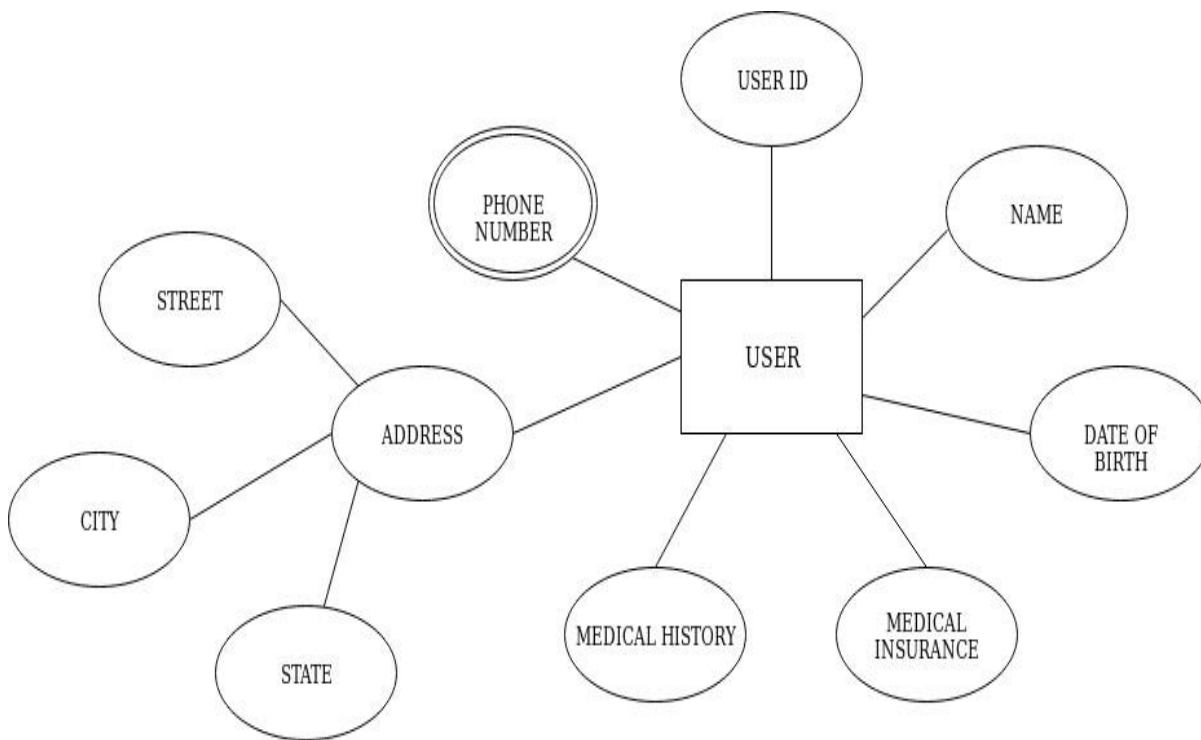
ORGANIZATION:



PATIENT:



USER:



3.2 SCHEMA DIAGRAM

DONAR

<u>DONAR_ID</u>	ORGAN_DONATION	REASON_OF_DONATION	ORGANIZATION_ID	USER_ID
-----------------	----------------	--------------------	-----------------	---------

ORGAN_AVAILABLE

<u>ORGAN_ID</u>	ORGAN_NAME	DONAR_ID
-----------------	------------	----------

TRANSACTION

<u>PATIENT_ID</u>	<u>ORGAN_ID</u>	<u>DONAR_ID</u>	DATE_OF_TRANSMISSION	STATUS
-------------------	-----------------	-----------------	----------------------	--------

ORGANIZATION_PHONE_NO

<u>ORGANIZATION_ID</u>	PHONE_NO
------------------------	----------

DOCTOR_PHONE_NO

<u>DOCTOR_ID</u>	PHONE_NO
------------------	----------

ORGANIZATION_HEAD

<u>ORGANIZATION_ID</u>	<u>EMPLOYEE_ID</u>	NAME	DATE_OF_JOINING	TERM_LENGTH
------------------------	--------------------	------	-----------------	-------------

LOGIN

USERNAME	<u>PASSWORD</u>
----------	-----------------

USER

<u>USER_ID</u>	NAME	DOB	MEDICAL_INSURANCE	MEDICAL_HISTORY	STREET	CITY	STATE
----------------	------	-----	-------------------	-----------------	--------	------	-------

USER_PHONE_NO

<u>USER_ID</u>	PHONE_NO
----------------	----------

ORGANIZATION

<u>ORG_ID</u>	ORG_NAME	LOCAL	GOVT
---------------	----------	-------	------

DOCTOR

<u>DOCT_ID</u>	DOCT_NAME	DEPT_NAME	ORG_ID
----------------	-----------	-----------	--------

PATIENT

<u>PATIENT_ID</u>	ORGAN_REQ	REASON & PROCUREMENT	<u>DOCTOR_ID</u>	<u>USER_ID</u>
-------------------	-----------	----------------------	------------------	----------------

CHAPTER-4

CODING

Python main file

Packages:

```
from flask import Flask,render_template,session,request,redirect,url_for,flash
import mysql.connector,hashlib
import matplotlib.pyplot as plt
import numpy as np
```

Database Connection:

```
mydb = mysql.connector.connect(
    host='localhost',
    user='k',
    password='123',
    database = 'dbms_project'
)
mycursor = mydb.cursor(buffered=True)
```

4.1 Creating Tables:

create database dbms_project;

use dbms_project;

1.Login:

```
CREATE TABLE login(username VARCHAR(20) NOT NULL, password VARCHAR(20)
NOT NULL);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 username	varchar(20)	utf8mb4_general_ci		No	None			 Change  Drop More
☐	2 password	varchar(20)	utf8mb4_general_ci		No	None			 Change  Drop More

2. User: CREATE TABLE User(User_ID int NOT NULL, Name varchar(20) NOT NULL, Date_of_Birth date NOT NULL, Medical_insurance int, Medical_history varchar(20), Street varchar(20), City varchar(20), State varchar(20), PRIMARY KEY (User_ID));

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 User_ID	int(11)			No	None			Change Drop More
☐	2 Name	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 Date_of_Birth	date			No	None			Change Drop More
☐	4 Medical_insurance	int(11)			Yes	NULL			Change Drop More
☐	5 Medical_history	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6 Street	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	7 City	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	8 State	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More

3. User_phone_no:

CREATE TABLE User_phone_no(User_ID int NOT NULL, phone_no varchar(15), FOREIGN KEY(User_ID) REFERENCES User(User_ID) ON DELETE CASCADE);

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 User_ID	int(11)			No	None			Change Drop More
☐	2 phone_no	varchar(15)	utf8mb4_general_ci		Yes	NULL			Change Drop More

4.Organization:

CREATE TABLE Organization(Organization_ID int NOT NULL,Organization_name varchar(20) NOT NULL,Location varchar(20),Government_approved int, # 0 or 1PRIMARY KEY(Organization_ID));

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Organization_ID	int(11)			No	None			Change Drop More
☐	2 Organization_name	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 Location	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 Government_approved	int(11)			Yes	NULL			Change Drop More

5.Doctor:

CREATE TABLE Doctor(Doctor_ID int NOT NULL, Doctor_Name varchar(20) NOT NULL, Department_Name varchar(20) NOT NULL, organization_ID int NOT NULL, FOREIGN KEY(organization_ID) REFERENCES Organization(organization_ID) ON DELETE CASCADE, PRIMARY KEY(Doctor_ID));

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Doctor_ID	int(11)			No	None			Change Drop More
☐	2 Doctor_Name	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 Department_Name	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 organization_ID	int(11)			No	None			Change Drop More

5. Patient:

```
CREATE TABLE Patient(
Patient_ID int NOT NULL,
organ_req varchar(20) NOT NULL,
reason_of_procurement varchar(20),
Doctor_ID int NOT NULL,
User_ID int NOT NULL,
FOREIGN KEY(User_ID) REFERENCES User(User_ID) ON DELETE CASCADE,
FOREIGN KEY(Doctor_ID) REFERENCES Doctor(Doctor_ID) ON DELETE CASCADE,
PRIMARY KEY(Patient_Id, organ_req));
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Patient_ID	int(11)			No	None			Change Drop More
☐	2 organ_req	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 reason_of_procurement	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 Doctor_ID	int(11)			No	None			Change Drop More
☐	5 User_ID	int(11)			No	None			Change Drop More

6. Donar:

```
CREATE TABLE Donor(
Donor_ID int NOT NULL,
organ_donated varchar(20) NOT NULL,
reason_of_donation varchar(20),
Organization_ID int NOT NULL,
User_ID int NOT NULL,
FOREIGN KEY(User_ID) REFERENCES User(User_ID) ON DELETE CASCADE,
FOREIGN KEY(Organization_ID) REFERENCES Organization(Organization_ID) ON DELETE CASCADE,
PRIMARY KEY(Donor_ID, organ_donated)
```


);

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Donor_ID	int(11)			No	None			Change Drop More
☐	2 organ_donated	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 reason_of_donation	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 Organization_ID	int(11)			No	None			Change Drop More
☐	5 User_ID	int(11)			No	None			Change Drop More

7. Organ_available:

```
CREATE TABLE Organ_available(  
Organ_ID int NOT NULL AUTO_INCREMENT,  
Organ_name varchar(20) NOT NULL,  
Donor_ID int NOT NULL,  
FOREIGN KEY(Donor_ID) REFERENCES Donor(Donor_ID) ON DELETE CASCADE,  
PRIMARY KEY(Organ_ID)  
);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Organ_ID	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 Organ_name	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 Donor_ID	int(11)			No	None			Change Drop More

8. Transaction:

```
CREATE TABLE Transaction(  
Patient_ID int NOT NULL, Organ_ID int NOT NULL,  
Donor_ID int NOT NULL, Date_of_transaction date NOT NULL,  
Status int NOT NULL,  
#0 or 1 FOREIGN KEY(Patient_ID) REFERENCES Patient(Patient_ID)  
ON DELETE CASCADE, FOREIGN KEY(Donor_ID)  
REFERENCES Donor(Donor_ID) ON DELETE CASCADE,  
PRIMARY KEY(Patient_ID, Organ_ID));
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Patient_ID	int(11)			No	None			Change Drop More
☐	2 Organ_ID	int(11)			No	None			Change Drop More
☐	3 Donor_ID	int(11)			No	None			Change Drop More
☐	4 Date_of_transaction	date			No	None			Change Drop More
☐	5 Status	int(11)			No	None			Change Drop More

9. Organization_phone_no:

```
CREATE TABLE Organization_phone_no(Organization_ID int NOT NULL,
Phone_no varchar(15),
FOREIGN KEY(Organization_ID) REFERENCES Organization(Organization_ID) ON DELETE
CASCADE
);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Organization_ID	int(11)			No	None			Change Drop More
☐	2 Phone_no	varchar(15)	utf8mb4_general_ci		Yes	NULL			Change Drop More

10. Doctor_phone_no:

```
CREATE TABLE Doctor_phone_no(
Doctor_ID int NOT NULL,
Phone_no varchar(15),
FOREIGN KEY(Doctor_ID) REFERENCES Doctor(Doctor_ID) ON DELETE CASCADE
);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Doctor_ID	int(11)			No	None			Change Drop More
☐	2 Phone_no	varchar(15)	utf8mb4_general_ci		Yes	NULL			Change Drop More

11. Organization_head:

```
CREATE TABLE Organization_head(
Organization_ID int NOT NULL,
Employee_ID int NOT NULL,
Name varchar(20) NOT NULL,
```

Date_of_joining date NOT NULL,

Term_length int NOT NULL,

FOREIGN KEY(Organization_ID) REFERENCES Organization(Organization_ID) ON DELETE CASCADE,

PRIMARY KEY(Organization_ID,Employee_ID)

);

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 Organization_ID	int(11)			No	None			Change Drop More
☐	2 Employee_ID	int(11)			No	None			Change Drop More
☐	3 Name	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 Date_of_joining	date			No	None			Change Drop More
☐	5 Term_length	int(11)			No	None			Change Drop More

12.Log:

create table log (

querytime datetime,

comment varchar(255)

);

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 querytime	datetime			Yes	NULL			Change Drop More
☐	2 comment	varchar(255)	utf8mb4_general_ci		Yes	NULL			Change Drop More

4.2Inserting Tables:

1.Login:

INSERT INTO login VALUES ('admin','admin');

username	password
admin	admin

2.User:

INSERT INTO User VALUES (1, 'John Doe', '1990-05-15', 1, 'No major issues', '123 Main St', 'Cityville', 'Stateville');

INSERT INTO User VALUES (2, 'Jane Smith', '1985-08-22', 0, 'None', '456 Oak St', 'Townsville', 'Stateville');

INSERT INTO User VALUES (3, 'Bob Johnson', '1978-12-10', 1, 'Allergies', '789 Pine St', 'Villagetown', 'Stateland');

INSERT INTO User VALUES (4, 'Alice Williams', '1995-03-27', 1, 'High blood pressure', '101 Cedar St', 'Hamletville', 'Stateville');

INSERT INTO User VALUES (5, 'Charlie Brown', '1980-06-14', 0, 'None', '202 Elm St', 'Ruraltown', 'Stateville');

INSERT INTO User VALUES (6, 'Eva Davis', '1992-09-03', 1, 'Asthma', '303 Birch St', 'Suburbia', 'Stateland');

INSERT INTO User VALUES (7, 'Frank Miller', '1987-04-18', 0, 'None', '404 Maple St', 'Cityburg', 'Stateland');

INSERT INTO User VALUES (8, 'Grace Wilson', '1975-11-09', 1, 'Diabetes', '505 Pine St', 'Hometown', 'Stateville');

INSERT INTO User VALUES (9, 'David Anderson', '1998-02-25', 0, 'None', '606 Oak St', 'Villageville', 'Stateland');

INSERT INTO User VALUES (10, 'Sophia Lee', '1983-07-08', 1, 'No major issues', '707 Cedar St', 'Countryside', 'Stateville');

				User_ID	Name	Date_of_Birth	Medical_insurance	Medical_history	Street	City	State
☐	 Edit	 Copy	 Delete	1	sneha	2003-05-28	109	none	dr.ait	Banglore	Karnataka
☐	 Edit	 Copy	 Delete	2	Jane Smith	1985-08-22	0	None	456 Oak St	Townsville	Stateville
☐	 Edit	 Copy	 Delete	3	Abhi	0000-00-00	0	None	G5	Banglore	Karnataka
☐	 Edit	 Copy	 Delete	4	Alice Williams	1995-03-27	1	High blood pressure	101 Cedar St	Hamletville	Stateville
☐	 Edit	 Copy	 Delete	5	Kushal	2003-07-26	0	None	101	Banglore	Karnataka
☐	 Edit	 Copy	 Delete	6	Eva Davis	1992-09-03	1	Asthma	303 Birch St	Suburbia	Stateland
☐	 Edit	 Copy	 Delete	7	Frank Miller	1987-04-18	0	None	404 Maple St	Cityburg	Stateland
☐	 Edit	 Copy	 Delete	8	Grace Wilson	1975-11-09	1	Diabetes	505 Pine St	Hometown	Stateville
☐	 Edit	 Copy	 Delete	9	David Anderson	1998-02-25	0	None	606 Oak St	Villageville	Stateland
☐	 Edit	 Copy	 Delete	10	Sophia Lee	1983-07-08	1	No major issues	707 Cedar St	Countryside	Stateville

3. user_phone_no:

INSERT INTO User_phone_no (User_ID, phone_no) VALUES (1, '123-456-7890');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (2, '987-654-3210');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (3, '555-123-4567');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (4, '789-012-3456');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (5, '111-222-3333');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (6, '444-555-6666');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (7, '999-888-7777');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (8, '123-789-4560');

INSERT INTO User_phone_no (User_ID, phone_no)VALUES (9, '555-876-5432');

```
INSERT INTO User_phone_no (User_ID, phone_no)VALUES (10, '987-654-1234');
```

User_ID	phone_no
2	987-654-3210
4	789-012-3456
6	444-555-6666
7	999-888-7777
8	123-789-4560
9	555-876-5432
10	987-654-1234
1	123-456-7890

4.Organization:

```
INSERT INTO Organization VALUES (1, 'Hospital A', 'City A', 1);
```

```
INSERT INTO Organization VALUES (2, 'Clinic B', 'City B', 0);
```

```
INSERT INTO Organization VALUES (3, 'Medical Center C', 'City C', 1);
```

```
INSERT INTO Organization VALUES (4, 'Healthcare Group D', 'City D', 1);
```

```
INSERT INTO Organization VALUES (5, 'Medical Services E', 'City E', 0);
```

```
INSERT INTO Organization VALUES (6, 'Hospital F', 'City F', 1);
```

```
INSERT INTO Organization VALUES (7, 'Clinic G', 'City G', 0);
```

```
INSERT INTO Organization VALUES (8, 'Healthcare Center H', 'City H', 1);
```

```
INSERT INTO Organization VALUES (9, 'Hospital I', 'City I', 1);
```

```
INSERT INTO Organization VALUES (10, 'Medical Group J', 'City J', 0);
```

			Organization_ID	Organization_name	Location	Government_approved
☐	 Edit	 Copy	 Delete	1 Hospital A	City A	1
☐	 Edit	 Copy	 Delete	2 Clinic B	City B	0
☐	 Edit	 Copy	 Delete	3 Medical Center C	City C	1
☐	 Edit	 Copy	 Delete	4 Healthcare Group D	City D	1
☐	 Edit	 Copy	 Delete	5 Medical Services E	City E	0
☐	 Edit	 Copy	 Delete	6 Hospital F	City F	1
☐	 Edit	 Copy	 Delete	7 Clinic G	City G	0
☐	 Edit	 Copy	 Delete	8 Healthcare Center H	City H	1
☐	 Edit	 Copy	 Delete	9 Hospital I	City I	1
☐	 Edit	 Copy	 Delete	10 Medical Group J	City J	0

5.Doctor:

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (1, 'Dr. Smith', 'Cardiology', 1);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (2, 'Dr. Johnson', 'Orthopedics', 2);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (3, 'Dr. Martinez', 'Neurology', 3);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (4, 'Dr. Turner', 'Pediatrics', 1);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (5, 'Dr. Harris', 'Oncology', 2);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (6, 'Dr. Clark', 'Dermatology', 3);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (7, 'Dr. White', 'Gastroenterology', 1);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (8, 'Dr. Turner', 'Cardiac Surgery', 2);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (9, 'Dr. Garcia', 'Plastic Surgery', 3);
```

```
INSERT INTO Doctor (Doctor_ID, Doctor_Name, Department_Name, organization_ID)
VALUES (10, 'Dr. Lee', 'Urology', 1);
```

		Organization_ID	Organization_name	Location	Government_approved
☐	 Edit  Copy  Delete	1	Hospital A	City A	1
☐	 Edit  Copy  Delete	2	Clinic B	City B	0
☐	 Edit  Copy  Delete	3	Medical Center C	City C	1
☐	 Edit  Copy  Delete	4	Healthcare Group D	City D	1
☐	 Edit  Copy  Delete	5	Medical Services E	City E	0
☐	 Edit  Copy  Delete	6	Hospital F	City F	1
☐	 Edit  Copy  Delete	7	Clinic G	City G	0
☐	 Edit  Copy  Delete	8	Healthcare Center H	City H	1
☐	 Edit  Copy  Delete	9	Hospital I	City I	1
☐	 Edit  Copy  Delete	10	Medical Group J	City J	0

6.Patient:

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (1, 'Heart', 'Transplant needed', 1, 1);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (2, 'Kidney', 'Chronic kidney disease', 2, 2);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (3, 'Liver', 'Cirrhosis', 3, 3);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (4, 'Lung', 'Idiopathic pulmonary fibrosis', 4, 4);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (5, 'Pancreas', 'Type 1 diabetes complications', 5, 5);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (6, 'Intestine', 'Short bowel syndrome', 6, 6);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (7, 'Cornea', 'Vision impairment', 7, 7);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (8, 'Bone Marrow', 'Leukemia treatment', 8, 8);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (9, 'Skin', 'Burn injuries', 9, 9);
```

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (10, 'Blood Vessel', 'Peripheral artery disease', 10, 10);
```

			Patient_ID	organ_req	reason_of_procurement	Doctor_ID	User_ID
☐		Edit		Copy		Delete	1 heart Slight heart attack 3 2
☐		Edit		Copy		Delete	2 Kidney Chronic kidney disea 2 2
☐		Edit		Copy		Delete	3 Eye sight 2 6
☐		Edit		Copy		Delete	4 Lung Idiopathic pulmonary 4 4
☐		Edit		Copy		Delete	5 heart heart problem 4 2
☐		Edit		Copy		Delete	6 Intestine Short bowel syndrome 6 6
☐		Edit		Copy		Delete	7 Cornea Vision impairment 7 7
☐		Edit		Copy		Delete	8 Bone Marrow Leukemia treatment 8 8
☐		Edit		Copy		Delete	9 Skin Burn injuries 9 9
☐		Edit		Copy		Delete	10 left eye accident 3 8

7.Organ_available:

```
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Heart', 1);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Kidney', 2);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Liver', 3);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Lung', 4);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Pancreas', 5);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Intestine', 6);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Cornea', 7);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Bone Marrow', 8);
INSERT INTO Organ_available (Organ_name, Donor_ID)VALUES ('Skin', 9);
```

				Organ_ID	Organ_name	Donor_ID
☐				11	Kidney	2
☐				13	Lung	4
☐				15	Intestine	6
☐				16	Cornea	7
☐				17	Bone Marrow	8
☐				18	Skin	9

8.Transaction:

```
INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (1, 1, 1, '2024-02-01', 1);
INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (2, 2, 2, '2024-02-15', 1);
INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (3, 3, 3, '2024-03-01', 0);
INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (4, 4, 4, '2024-03-15', 0);
INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (5, 5, 5, '2024-04-01', 1);
```



```

INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (6, 6, 6, '2024-04-15', 1);

INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (7, 7, 7, '2024-05-01', 0);

INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (8, 8, 8, '2024-05-15', 0);

INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (9, 9, 9, '2024-06-01', 1);

INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (10, 10, 10, '2024-06-15', 1);

```

←T→				Patient_ID	Organ_ID	Donor_ID	Date_of_transaction	Status
☐				2	2	2	2024-02-15	1
☐				4	4	4	2024-03-15	0
☐				6	6	6	2024-04-15	1
☐				7	7	7	2024-05-01	0
☐				8	8	8	2024-05-15	0
☐				9	9	9	2024-06-01	1

9.Organization_phone_no:

```

INSERT INTO Organization_phone_no (Organization_ID, Phone_no)
VALUES (1, '123-456-7890');

INSERT INTO Organization_phone_no (Organization_ID, Phone_no)
VALUES (2, '987-654-3210');

INSERT INTO Organization_phone_no (Organization_ID, Phone_no)
VALUES (3, '555-123-4567');

INSERT INTO Organization_phone_no (Organization_ID, Phone_no)
VALUES (4, '789-012-3456');

INSERT INTO Organization_phone_no (Organization_ID, Phone_no)

```

```
VALUES (5, '111-222-3333');
```

```
INSERT INTO Organization_phone_no (Organization_ID, Phone_no)
```

```
VALUES (6, '444-555-6666');
```

Organization_ID	Phone_no
1	123-456-7890
2	987-654-3210
3	555-123-4567
4	789-012-3456
5	111-222-3333
6	444-555-6666

10.Doctor_phone_no:

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (1, '123-456-7890');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (2, '987-654-3210');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (3, '555-123-4567');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (4, '789-012-3456');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (5, '111-222-3333');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (6, '444-555-6666');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (7, '999-888-7777');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (8, '123-789-4560');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (9, '555-876-5432');
```

```
INSERT INTO Doctor_phone_no (Doctor_ID, Phone_no)
```

```
VALUES (10, '987-654-1234');
```

Doctor_ID	Phone_no
1	123-456-7890
2	987-654-3210
3	555-123-4567
4	789-012-3456
5	111-222-3333
6	444-555-6666
7	999-888-7777
8	123-789-4560
9	555-876-5432
10	987-654-1234

11.Organization Head:

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (1, 103, 'Michael Brown', '2023-11-28', 2);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (2, 203, 'Sophie Miller', '2024-03-05', 3);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (3, 301, 'David Lee', '2024-02-20', 2);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (4, 401, 'Emma Turner', '2023-12-10', 1);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (3, 302, 'Oliver White', '2024-01-15', 3);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (5, 501, 'Grace Harris', '2024-03-10', 2);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (4, 402, 'Daniel Clark', '2023-11-15', 1);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

VALUES (5, 502, 'Ava Garcia', '2024-01-05', 2);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining, Term_length)

```
VALUES (1, 104, 'Christopher Taylor', '2023-12-20', 3);

INSERT INTO Organization_head (Organization_ID, Employee_ID, Name, Date_of_joining,
Term_length)

VALUES (6, 601, 'Lily Turner', '2024-02-25', 1);
```

						Organization_ID	Employee_ID	Name	Date_of_joining	Term_length
Edit Copy Delete		1	103	Michael Brown	2023-11-28	2				
Edit Copy Delete		1	104	Christopher Taylor	2023-12-20	3				
Edit Copy Delete		2	203	Sophie Miller	2024-03-05	3				
Edit Copy Delete		3	301	David Lee	2024-02-20	2				
Edit Copy Delete		3	302	Oliver White	2024-01-15	3				
Edit Copy Delete		4	401	Emma Turner	2023-12-10	1				
Edit Copy Delete		4	402	Daniel Clark	2023-11-15	1				
Edit Copy Delete		5	501	Grace Harris	2024-03-10	2				
Edit Copy Delete		5	502	Ava Garcia	2024-01-05	2				
Edit Copy Delete		6	601	Lily Turner	2024-02-25	1				

4.3 Queries:

1. Retrieve all patients along with their corresponding doctor's names and contact Numbers.

```
SELECT
P.Patient_ID,
U.Name AS Patient_Name,
D.Doctor_Name,
DP.phone_no AS Doctor_Phone_No
FROM
Patient P
JOIN User U ON P.User_ID = U.User_ID
JOIN Doctor D ON P.Doctor_ID = D.Doctor_ID
JOIN Doctor_phone_no DP ON D.Doctor_ID = DP.Doctor_ID;
```

Patient_ID	Patient_Name	Doctor_Name	Doctor_Phone_No
2	Jane Smith	Dr. Johnson	987-654-3210
3	Eva Davis	Dr. Johnson	987-654-3210
1	Jane Smith	Dr. Martinez	555-123-4567
10	Grace Wilson	Dr. Martinez	555-123-4567
4	Alice Williams	Dr. Turner	789-012-3456
5	Jane Smith	Dr. Turner	789-012-3456
6	Eva Davis	Dr. Clark	444-555-6666
7	Frank Miller	Dr. White	999-888-7777
8	Grace Wilson	Dr. Turner	123-789-4560
9	David Anderson	Dr. Garcia	555-876-5432

2. Retrieve the names and phone numbers of doctors in the 'Cardiology' department:

```
SELECT
D.Doctor_Name,
DP.Phone_no
FROM
Doctor D
JOIN Doctor_phone_no DP ON D.Doctor_ID = DP.Doctor_ID
WHERE
```

```
D.Department_Name = 'Cardiology';
```

Doctor_Name	Phone_no
Dr. Smith	123-456-7890

3. Update the reason for organ donation for Donor_ID 3:

```
UPDATE Donor
SET reason_of_donation = 'Personal choice and family history'
WHERE Donor_ID = 3;
```

✓ 0 rows affected. (Query took 0.0010 seconds.)

```
UPDATE Donor SET reason_of_donation = 'Personal choice and family history' WHERE Donor_ID = 3;
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

4. Retrieve the organizations and the total number of patients associated with each:

```
SELECT
O.Organization_name,
COUNT(P.Patient_ID) AS Total_Patients
FROM
Organization O
LEFT JOIN Doctor D ON O.Organization_ID = D.Organization_ID
LEFT JOIN Patient P ON D.Doctor_ID = P.Doctor_ID
```

GROUP BY O.Organization_name;

Organization_name	Total_Patients
Clinic B	3
Clinic G	0
Healthcare Center H	0
Healthcare Group D	0
Hospital A	3
Hospital F	0
Hospital I	0
Medical Center C	4
Medical Group J	0
Medical Services E	0

5. Insert a new patient record along with a transaction for a heart transplant:

```
INSERT INTO Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID)
VALUES (11, 'Heart', 'Critical condition', , 11);
```

```
INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status)
VALUES (11, 11, 1, '2024-07-01', 1);
```

✓ 1 row inserted. (Query took 0.0004 seconds.)

-- Assuming User_ID 11 and Doctor_ID 11 are valid **INSERT INTO** Patient (Patient_ID, organ_req, reason_of_procurement, Doctor_ID, User_ID) **VALUES** (11, 'Heart', 'Critical condition', 3, 1);

[Edit inline](#) | [Edit](#) | [Create PHP code](#)

✓ 1 row inserted. (Query took 0.0004 seconds.)

INSERT INTO Transaction (Patient_ID, Organ_ID, Donor_ID, Date_of_transaction, Status) **VALUES** (11, 10, 10, '2024-07-01', 1);

[Edit inline](#) | [Edit](#) | [Create PHP code](#)

6.Retrieve the names and reasons for donation for donors associated with organizations

approved by the government:

SELECT

U.Name AS Donor_Name,

	 Patient_ID	Organ_ID	Donor_ID	Date_of_transaction	Status
  Edit  Copy  Delete	2	2	2	2024-02-15	1
  Edit  Copy  Delete	4	4	4	2024-03-15	0
  Edit  Copy  Delete	6	6	6	2024-04-15	1
  Edit  Copy  Delete	7	7	7	2024-05-01	0
  Edit  Copy  Delete	8	8	8	2024-05-15	0
  Edit  Copy  Delete	9	9	9	2024-06-01	1
  Edit  Copy  Delete	11	10	10	2024-07-01	1

	 Patient_ID	organ_req	reason_of_procurement	Doctor_ID	User_ID
  Edit  Copy  Delete	1	heart	Slight heart attack	3	2
  Edit  Copy  Delete	2	Kidney	Chronic kidney disea	2	2
  Edit  Copy  Delete	3	Eye	sight	2	6
  Edit  Copy  Delete	4	Lung	Idiopathic pulmonary	4	4
  Edit  Copy  Delete	5	heart	heart problem	4	2
  Edit  Copy  Delete	6	Intestine	Short bowel syndrome	6	6
  Edit  Copy  Delete	7	Cornea	Vision impairment	7	7
  Edit  Copy  Delete	8	Bone Marrow	Leukemia treatment	8	8
  Edit  Copy  Delete	9	Skin	Burn injuries	9	9
  Edit  Copy  Delete	10	left eye	accident	3	8
  Edit  Copy  Delete	11	Heart	Critical condition	3	1

D.reason_of_donation

FROM

Donor D

JOIN User U ON D.User_ID = U.User_ID

JOIN Organization O ON D.Organization_ID = O.Organization_ID

WHERE

O.Government_approved = 1;

Donor_Name	reason_of_donation
Alice Williams	Gratitude for life
Eva Davis	Volunteer for resear
Frank Miller	Helping restore visi
David Anderson	Contributing to burn
Sophia Lee	Improving vascular h

7. Update the date of birth for User_ID 2 to '1985-08-23':

UPDATE User


```
SET Date_of_Birth = '1985-08-23'
```

```
WHERE
```

```
User_ID = 2;
```

✓ 1 row affected. (Query took 0.0005 seconds.)

```
UPDATE User SET Date_of_Birth = '1985-08-23' WHERE User_ID = 2;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

				User_ID	Name	Date_of_Birth	Medical_insurance	Medical_history	Street	City	State
☐	 Edit	 Copy	 Delete	1	sneha	2003-05-28	109	none	dr.ait	Banglore	Karnataka
☐	 Edit	 Copy	 Delete	2	Jane Smith	1985-08-23	0	None	456 Oak St	Townsville	Stateville

8. Retrieve the names of patients who have undergone a transplant, along with the corresponding doctor's name and organization:

```
SELECT
```

```
U.Name AS Patient_Name,
```

```
D.Doctor_Name,
```

```
O.Organization_name
```

```
FROM
```

```
Patient P
```

```
JOIN User U ON P.User_ID = U.User_ID
```

```
JOIN Transaction T ON P.Patient_ID = T.Patient_ID
```

```
JOIN Doctor D ON P.Doctor_ID = D.Doctor_ID
```

```
JOIN Organization O ON D.Organization_ID = O.Organization_ID
```

```
WHERE
```

```
T.Status = 1;
```

Patient_Name	Doctor_Name	Organization_name
Jane Smith	Dr. Johnson	Clinic B
Eva Davis	Dr. Clark	Medical Center C
David Anderson	Dr. Garcia	Medical Center C
sneha	Dr. Martinez	Medical Center C

9. Retrieve the organizations that have at least one patient who has undergone a transplant,

and display the total number of such patients for each organization:

```
SELECT
    O.Organization_name,
    COUNT(DISTINCT P.Patient_ID) AS Total_Transplanted_Patients
FROM
    Organization O
    LEFT JOIN Doctor D ON O.Organization_ID = D.Organization_ID
    LEFT JOIN Patient P ON D.Doctor_ID = P.Doctor_ID
    LEFT JOIN Transaction T ON P.Patient_ID = T.Patient_ID
WHERE
    T.Status = 1
GROUPBY O.Organization_name;
```

Organization_name	Total_Transplanted_Patients
Clinic B	1
Medical Center C	3

10. Retrieve the names and details of patients who have not yet undergone a transplant

and have a medical history:

```
SELECT
    U.Name AS Patient_Name,
    U.Date_of_Birth,
    U.Medical_history
FROM
    User U
    LEFT JOIN Patient P ON U.User_ID = P.User_ID
    LEFT JOIN Transaction T ON P.Patient_ID = T.Patient_ID
WHERE
    P.Patient_ID IS NULL OR T.Status IS NULL;
```

Patient_Name	Date_of_Birth	Medical_history
Jane Smith	1985-08-23	None
Jane Smith	1985-08-23	None
Abhi	0000-00-00	None
Kushal	2003-07-26	None
Eva Davis	1992-09-03	Asthma
Grace Wilson	1975-11-09	Diabetes
Sophia Lee	1983-07-08	No major issues

CHAPTER 5

CODING:

```
from flask import Flask,render_template,session,request,redirect,url_for,flash
import mysql.connector,hashlib
import matplotlib.pyplot as plt
import numpy as np

mydb = mysql.connector.connect(
    host='localhost',
    user='root',
    password="",
    database = 'dbms_project'
)
mycursor = mydb.cursor(buffered=True)

app = Flask(__name__)

@app.route("/",methods = ['POST', 'GET'])
@app.route("/home",methods = ['POST','GET'])
def home():
    if not session.get('login'):
        return render_template('login.html'),401
    else:
        if session.get('isAdmin') :
            return render_template('home.html',username=session.get('username'))
        else :
            return home_student()

@app.route("/login",methods = ['GET','POST'])
def login():
    if request.method=='POST' :
        query = """SELECT * FROM login WHERE username = '%s'""" %(request.form['username'])
```

```

mycursor.execute(query)
res = mycursor.fetchall()
if mycursor.rowcount == 0:
    return home()
if request.form['password'] != res[0][1]:
    return render_template('login.html')
else:
    session['login'] = True
    session['username'] = request.form['username']
    session['password'] = request.form['password']
    session['isAdmin'] = (request.form['username']=='admin')
    return home()
return render_template('login.html')

```

```
@app.route("/show_update_detail",methods=['POST','GET'])
```

```

def show_update_detail():
    if not session.get('login'):
        return redirect( url_for('home') )
    if request.method=='POST':
        if request.form['User_ID'] == "":
            return render_template("search_detail.html")
        qry = "Select * from User where User.User_ID = %s" %(request.form['User_ID'])
        qry1 = "Select * from User_phone_no where User_ID = %s" %(request.form['User_ID'])
        mycursor.execute(qry)
        not_found=False
        res=()
        if(mycursor.rowcount > 0):
            res = mycursor.fetchone()
        else:
            not_found=True
        fields = mycursor.column_names
        qry_upd = "Select * from User where User_ID = %s" %(request.form['User_ID'])

```

```

mycursor.execute(qry_upd)

upd_res = ()

if(mycursor.rowcount > 0):
    upd_res = mycursor.fetchone()

fields_upd = mycursor.column_names

mycursor.execute(qry1)

phone_no = mycursor.fetchall()

qry_pat = "select Patient_ID, organ_req, reason_of_procurement, Doctor_name from Patient
inner join Doctor on Doctor.Doctor_ID = Patient.Doctor_ID and User_ID = %s"
%(request.form['User_ID'])

qry_don = "select Donor_ID, organ_donated, reason_of_donation, Organization_name from
Donor inner join Organization on Organization.Organization_ID = Donor.Organization_ID and
User_ID = %s" %(request.form['User_ID'])

qry_trans = "select distinct Transaction.Patient_ID, Transaction.Donor_ID, Organ_ID,
Date_of_transaction, Status from Transaction, Patient, Donor where (Patient.User_ID = %s and
Patient.Patient_ID = Transaction.Patient_ID) or (Donor.User_Id= %s and Donor.Donor_ID =
Transaction.Donor_ID)" %((request.form['User_ID']),(request.form['User_ID']))

#

res_pat = ()

res_dnr = ()

res_trans = ()

mycursor.execute(qry_pat)

if(mycursor.rowcount > 0):
    res_pat = mycursor.fetchall()

fields_pat = mycursor.column_names

#

mycursor.execute(qry_don)

if(mycursor.rowcount > 0):
    res_dnr = mycursor.fetchall()

fields_dnr = mycursor.column_names

#

mycursor.execute(qry_trans)

if(mycursor.rowcount > 0):
    res_trans = mycursor.fetchall()

fields_trans = mycursor.column_names

```

```

print(res_trans)

if("show" in request.form):

    return render_template('show_detail_2.html',res = res,fields = fields, not_found=not_found,
phone_no = phone_no, res_dnr = res_dnr, res_pat = res_pat,res_trans = res_trans,fields_trans =
fields_trans, fields_dnr = fields_dnr, fields_pat = fields_pat)

if("update" in request.form):

    return render_template('update_detail.html',res = upd_res,fields = fields_upd,
not_found=not_found)

if "delete" in request.form:

    if not_found:

        return render_template('show_detail_2.html',res = res,fields = fields,
not_found=not_found, phone_no = phone_no, res_dnr = res_dnr, res_pat = res_pat,res_trans =
res_trans,fields_trans = fields_trans, fields_dnr = fields_dnr, fields_pat = fields_pat)

    else:

        qry2 = "DELETE FROM User where User_ID = %s" %(request.form["User_ID"])
        mycursor.execute(qry2)
        mydb.commit()
        return render_template("home.html")

@app.route("/search_detail",methods = ['POST','GET'])
def search_detail():

    if not session.get('login'):

        return redirect( url_for('home') )

    return render_template('search_detail.html')

```

CHAPTER 6

SNAPSHOTS

Login Page:

Organ Donation and Procurement Management System

Username

admin

Password

.....

Submit

Home Page:

HOME

Hi admin!

Logout

Organ Donation and Procurement Management System

[User](#)

[Search](#)

[Add](#)

[Remove](#)

[Statistics](#)

User Page:

User

View/Delete User Details

View Page:

USER DETAILS		
1	User_ID	1
2	Name	sneha
3	Date_of_Birth	2003-05-28
4	Medical_insurance	109
5	Medical_history	none
6	Street	dr.ait
7	City	Banglore
8	State	Karnataka
17	Phone Numbers	123-456-7890 ;

Search Page:

Search

Search User by Details

Search Patient by Details

Search Donor by Details

Search Organ by Details

Search Organization by Details

Search Organization Head by Details

Search Doctor Details

Search Transaction Details

Search Log Details

Search

Search User by Details

Search Patient by Details

Search Donor by Details

Search Organ by Details

Search Organization by Details

Search Organization Head by Details

Search Doctor Details

Search Transaction Details

Search Log Details

Search patient details:

Organ Donation and Procurement Management System

SEARCH:

Patient_ID	organ_req	reason_of_procurement	Doctor_ID	User_ID
5	heart	heart problem	4	2

Add Page:

[Add](#)

Add User

Add User Phone Number

Add Patient

Add Donor

Add Doctor

Add Doctor Phone Number

Add Organization

Add Organization Phone Number

Add Organization Head

Add Transaction

Add Doctor Page:

Organ Procurement and Donation Management System

Successfully Added Doctor

Doctor_ID	12
Doctor_Name	Sneha BN
Department_Name	Cardiologist
organization_ID	5

Add Doctor

Remove Page:

[Remove](#)

Remove User

Remove Patient

Remove Donor

Remove Doctor

Remove Organization

Remove Organization Head

Remove Organization Page:

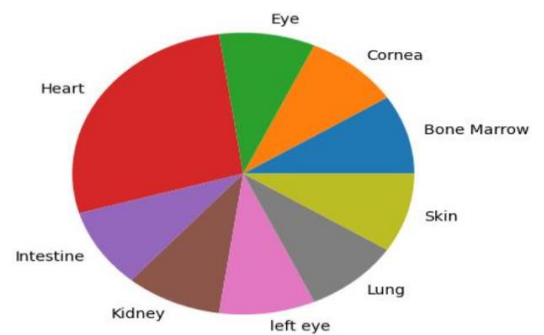
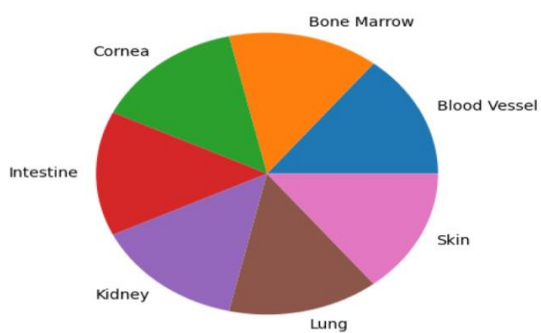
Organization ID

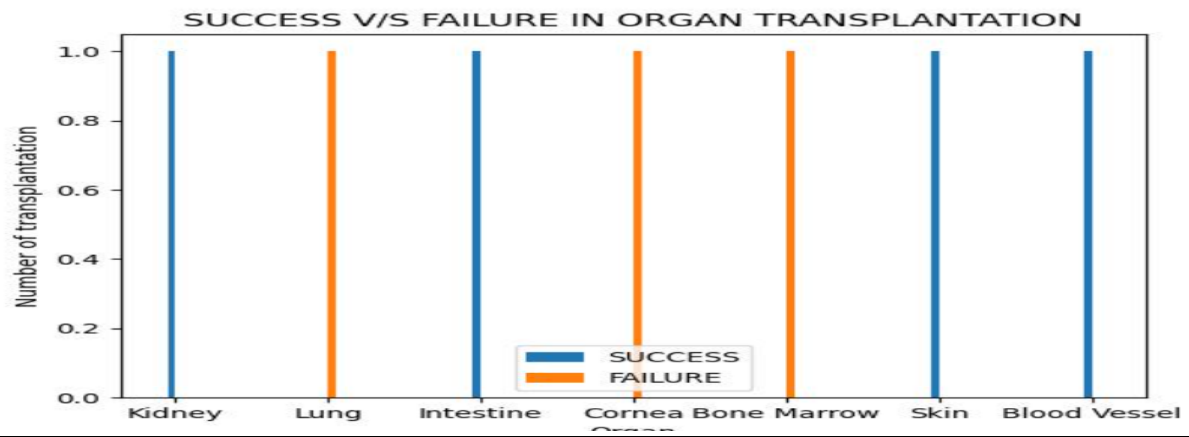
24

Delete Organization

Statistics Page:

STATISTICS





CONCLUSION

This project plays a vital role in organizing and helping the organ donation community by maintaining the record of donors. Thus we have successfully implemented organ donation database management which helps us in centralizing the data used for managing the tasks performed in a organ donation we have successfully implemented various functionalities of MY SQL and PHP and created the fully functional database management system for organ donation. MYSQL The database system used is MySQL which is an open source RDBMS. It is very light and highly functional.

It's all about today's generation, the whole world is with you in in your hand. This project informs about necessary support and provides critical recommendations for organ donation by mobile users who have to install the application or use by website when in need. This management will save many lives of people who are suffering due to lack of organ. This project will impact and address to society and humanity with spreading the concept of life after one's death.

This management will give the most user-friendly platform to sign up both the organ donor user or the organ receiver in such an instant and organized way. We mainly focusing to give information in order to prepare organ donor/receiver for this system when patients are in need of organ and who International Journal of Computer Science, Engineering and Applications are ready to donate organ on their wish. This all can bring out the positive outcome.

Therefore, we should spread awareness and encourage the acceptance of organ donation management system by using mobile or website in health care system to save lives as much as possible. We acknowledge the importance of every specific person of team work while project development and management.

MYSQL The database system used is MySQL which is an open source RDBMS. It is very light and highly functional.

BIBLIOGRAPHY

Books:

- Ramez Elmasri and Shamkant B Navathe, "Fundamental Of Database System", Pearson Education Limited, Seventh Edition, 2015.

Websites:

- <https://dev.mysql.com/doc/>
- <https://dev.mysql.com/doc/refman/8.0/en/>
- <https://www.w3schools.com/mysql/>
- <https://www.javatpoint.com/dbms-tutorial>
- <https://github.com/tarun360>